

Task 1

Temperature Comparison Script

Write a Python script that sends the same prompt to an LLM API three times with different temperature values (0.1, 0.5, 1.0) and prints all three responses side by side.

Requirements:

- Use OpenAI or any available API
- Loop over temperature values [0.1, 0.5, 1.0]
- Print the temperature value before each response
- Observe and comment how output changes with temperature

[Temperature.py](#)

```
import os
from pathlib import Path

from openai import OpenAI
from dotenv import load_dotenv

# Load the project key file before creating the client.
load_dotenv(Path(__file__).parent / ".venv" / ".env",
            override=True)

# Initialize the OpenAI client (reads OPENAI_API_KEY from
environment)
client = OpenAI()

# Prompt designed to test creative variation
PROMPT = "Write a 3-sentence story about a robot discovering a
flower on a desolate planet."
TEMPERATURES = [0.1, 0.5, 1.0]

results = {}
```

```

# Iterate over temperature settings
for temp in TEMPERATURES:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "user", "content": PROMPT}
        ],
        temperature=temp,
    )
    # Store response text
    results[temp] = response.choices[0].message.content.strip()

# Print formatted side-by-side comparison
print("=" * 80)
print(f"PROMPT: {PROMPT}")
print("=" * 80)

for temp, text in results.items():
    print(f"\n--- TEMPERATURE: {temp} ---")
    print(text)

print("\n" + "=" * 80)

```

Output python3 [temperature.py](#)

```
(venv) appleAPPLEs-MacBook-Air temperature % cat temperature.py
import os
from pathlib import Path

from openai import OpenAI
from dotenv import load_dotenv

# Load the project key file before creating the client.
load_dotenv(Path(__file__).parent / ".venv" / ".env", override=True)

# Initialize the OpenAI client (reads OPENAI_API_KEY from environment)
client = OpenAI()

# Prompt designed to test creative variation
PROMPT = "Write a 3-sentence story about a robot discovering a flower on a desolate planet."
TEMPERATURES = [0.1, 0.5, 1.0]

results = {}

# Iterate over temperature settings
for temp in TEMPERATURES:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "user", "content": PROMPT}
        ],
        temperature=temp,
    )
    # Store response text
    results[temp] = response.choices[0].message.content.strip()

# Print formatted side-by-side comparison
print("\n" * 80)
print(f"===== (PROMPT) =====")
print("\n" * 80)

for temp, text in results.items():
    print(f"----- TEMPERATURE: {temp} -----")
    print(text)

print("\n" * 80)
(venv) appleAPPLEs-MacBook-Air temperature % python3 temperature.py
=====
PROMPT: Write a 3-sentence story about a robot discovering a flower on a desolate planet.

----- TEMPERATURE: 0.1 -----
As the robot roamed the barren landscape of the desolate planet, its sensors detected a faint pulse of color amidst the gray dust. Curiosity piqued, it approached to find a delicate flower, vibrant and resilient, defiantly blooming in the harsh environment. In that moment, the robot felt an unexpected spark of wonder, realizing that even in the bleakest places, life could still find a way to thrive.

----- TEMPERATURE: 0.5 -----
As the robot roamed the barren landscape of the desolate planet, its sensors detected an unusual signal emanating from beneath a layer of dust. Curiously, it unearthed a vibrant flower, its petals shimmering with colors unseen in the mechanical world. In that moment, the robot felt a spark of wonder, realizing that even in the harshest environments, life could find a way to bloom.

----- TEMPERATURE: 1.0 -----
As the dust settled around its metallic feet, the robot scanned the barren landscape of the desolate planet, its sensors detecting an anomaly. Among the cracked soil and jagged rocks, a vibrant flower, defiantly blooming, caught its attention, petals shimmering like jewels against the stark backdrop. In that moment, the robot felt an unfamiliar warmth in its circuits, realizing that even in the harshest environments, beauty could exist against all odds.
```

The screenshot shows a code editor with a file explorer on the left, a code editor in the center, and a terminal at the bottom. The file explorer shows a project structure with a file named `temperature.py`. The code editor displays the Python code for `temperature.py`, which uses the OpenAI API to generate three sentences of text for different temperature settings (0.1, 0.5, and 1.0). The terminal shows the output of the script, which is a 3-sentence story about a robot discovering a flower on a desolate planet, generated for each temperature setting.

```
temperature.py
1 import os
2 from pathlib import Path
3
4 from openai import OpenAI
5 from dotenv import load_dotenv
6
7 # Load the project key file before creating the client.
8 load_dotenv(Path(__file__).parent / ".venv" / ".env", override=True)
9
10 # Initialize the OpenAI client (reads OPENAI_API_KEY from environment)
11 client = OpenAI()
12
13 # Prompt designed to test creative variation
14 PROMPT = "Write a 3-sentence story about a robot discovering a flower on a desolate planet."
15 TEMPERATURES = [0.1, 0.5, 1.0]
16
17 results = {}
18
19 # Iterate over temperature settings
20 for temp in TEMPERATURES:
21     response = client.chat.completions.create(
22         model="gpt-4o-mini",
23         messages=[
24             {"role": "user", "content": PROMPT}
25         ],
26         temperature=temp,
27     )
28     # Store response text
29     results[temp] = response.choices[0].message.content.strip()
30
31 # Print formatted side-by-side comparison
32 print("\n" * 80)
33 print(f"===== (PROMPT) =====")
34 print("\n" * 80)
35
36 for temp, text in results.items():
37     print(f"----- TEMPERATURE: {temp} -----")
38     print(text)
39
40 print("\n" * 80)
```

```
(.venv) appleAPPLEs-MacBook-Air temperature % python3 temperature.py
=====
PROMPT: Write a 3-sentence story about a robot discovering a flower on a desolate planet.

----- TEMPERATURE: 0.1 -----
As the robot roamed the barren landscape of the desolate planet, its sensors detected a faint pulse of color amidst the gray dust. Curiosity piqued, it approached the delicate flower, its vibrant petals shimmering like jewels against the stark surroundings. In that moment, the robot felt an unexpected spark of wonder, realizing that even in the bleakest of places, beauty could still bloom.

----- TEMPERATURE: 0.5 -----
As the lonely robot traversed the barren landscape of the desolate planet, its sensors detected a faint pulse of color amidst the gray dust. Curiosity sparked in its circuits as it approached the delicate flower, vibrant and resilient, defiantly blooming in the harsh environment. In that moment, the robot felt an unexpected surge of hope, realizing that even in the bleakest places, life could find a way to thrive.

----- TEMPERATURE: 1.0 -----
Amidst the barren landscapes of a desolate planet, a solitary robot trudged through the cracked earth, its sensors scanning for signs of life. Suddenly, a flash of color caught its attention—a vibrant flower, resiliently blooming in the harsh environment, its petals glistening with dewdrops. In that moment, the robot's programming shifted, and it felt an unfamiliar spark of wonder, realizing that beauty could thrive even in the most unexpected places.
```

Task 2

Schedule-Triggered AI Summary Workflow

Build an n8n workflow that uses a Manual Trigger to pass a hardcoded topic to an OpenAI node and returns a 3-bullet-point summary of that topic.

Requirements:

- Use a Manual Trigger node to start the workflow
- Pass a fixed topic string to the OpenAI node via a Set node
- Prompt the LLM: 'Give 3 bullet points about: {topic}'
- Display output in n8n execution results panel

```
{
  "nodes": [
    {
      "parameters": {},
      "id": "1",
      "name": "When clicking 'Test workflow'",
      "type": "n8n-nodes-base.manualTrigger",
      "typeVersion": 1,
      "position": [220, 300]
    },
    {
      "parameters": {
        "assignments": {
          "assignments": [
            {
              "id": "set-topic-id",
              "name": "topic",
              "value": "Artificial Intelligence in Healthcare",
              "type": "string"
            }
          ]
        },
        "options": {}
      },
      "id": "2",
      "name": "Set Topic",
      "type": "n8n-nodes-base.set",
      "typeVersion": 3.4,
      "position": [440, 300]
    },
    {
      "parameters": {
        "resource": "text",
        "operation": "complete",
        "model": "gpt-4o-mini",
        "prompt": "Give 3 bullet points about: {{ $json.topic }}",
        "options": {}
      },
      "id": "3",
      "name": "OpenAI",
      "type": "n8n-nodes-base.openAi",

```

```

    "typeVersion": 1,
    "position": [660, 300],
    "credentials": {
      "openAiApi": {
        "id": "YOUR_OPENAI_CREDENTIAL_ID",
        "name": "OpenAI account"
      }
    }
  },
  "connections": {
    "When clicking 'Test workflow'": {
      "main": [
        [
          {
            "node": "Set Topic",
            "type": "main",
            "index": 0
          }
        ]
      ]
    },
    "Set Topic": {
      "main": [
        [
          {
            "node": "OpenAI",
            "type": "main",
            "index": 0
          }
        ]
      ]
    }
  }
}

```

Task 3

Simple End-to-End RAG Pipeline

Build a Python script that loads a text file, chunks and embeds its content into a vector store, accepts a user question, retrieves relevant chunks, and passes them to an LLM to generate a grounded answer.

Requirements:

- Use any text document (e.g., a Wikipedia article saved as .txt)
- Use Chroma or Pinecone + OpenAI/Ollama embeddings

- Show the retrieved context chunks before the final answer
- Accept user questions in a loop (at least 3 questions)

[RAG.py](#)

```
import os
import sys
from importlib import import_module

def _import_symbol(module_name, symbol_name):
    try:
        return getattr(import_module(module_name), symbol_name)
    except (ImportError, AttributeError) as exc:
        raise RuntimeError(
            "Missing RAG dependencies. Install them with: "
            "python -m pip install python-dotenv\n"
            "langchain-community\n"
            "langchain-text-splitters langchain-openai\n"
            "langchain-chroma"
        ) from exc

# Load dependencies dynamically so static analyzers do not
# report unresolved
# imports when this file is opened outside the project's
# configured environment.
try:
    load_dotenv = _import_symbol("dotenv", "load_dotenv")
    TextLoader =
    _import_symbol("langchain_community.document_loaders",
    "TextLoader")
    RecursiveCharacterTextSplitter = _import_symbol(
```

```

        "langchain_text_splitters",
"RecursiveCharacterTextSplitter"
    )
    OpenAIEmbeddings = _import_symbol("langchain_openai",
"OpenAIEmbeddings")
    ChatOpenAI = _import_symbol("langchain_openai",
"ChatOpenAI")
    Chroma = _import_symbol("langchain_chroma", "Chroma")
    ChatPromptTemplate =
_import_symbol("langchain_core.prompts", "ChatPromptTemplate")
except RuntimeError as exc:
    print(f"Error: {exc}", file=sys.stderr)
    sys.exit(1)

# 1. Load Environment Variables
load_dotenv(override=True)

if not os.getenv("OPENAI_API_KEY"):
    print("Error: OPENAI_API_KEY environment variable is not
set.")
    sys.exit(1)

# 2. Setup Sample Document (or pass an existing .txt file)
DOCUMENT_PATH = "sample_data.txt"

def create_sample_document():
    """Helper to generate a dummy document if one doesn't
exist."""
    if not os.path.exists(DOCUMENT_PATH):
        sample_text = ""
        The Apollo 11 mission was the spaceflight that first
landed humans on the Moon.

```

Commander Neil Armstrong and Lunar Module Pilot Buzz Aldrin landed the Apollo Lunar Module Eagle on July 20, 1969.

Neil Armstrong became the first person to step onto the lunar surface six hours and 39 minutes later on July 21 at 02:56 UTC.

Buzz Aldrin joined him 19 minutes later. They spent about two and a quarter hours together outside the spacecraft.

They collected 47.5 pounds (21.5 kg) of lunar material to bring back to Earth.

Command Module Pilot Michael Collins flew the Command Module Columbia alone in lunar orbit while they were on the Moon's surface.

Armstrong and Aldrin spent 21 hours, 36 minutes on the lunar surface at a site they named Tranquility Base.

The mission was launched by a Saturn V rocket from Kennedy Space Center in Merritt Island, Florida.

```
""  
  
with open(DOCUMENT_PATH, "w", encoding="utf-8") as f:  
    f.write(sample_text.strip())  
    print(f"[+] Created sample document at  
'{DOCUMENT_PATH}'")
```

```
# 3. Build RAG Pipeline
```

```
def initialize_rag_pipeline(file_path: str):  
    print("\n--- Loading Document & Indexing Vector Store ---")  
  
    # Load raw text document  
    loader = TextLoader(file_path, encoding="utf-8")  
    documents = loader.load()  
  
    # Split text into manageable chunks  
    text_splitter = RecursiveCharacterTextSplitter(  
        chunk_size=300,
```

```

        chunk_overlap=50
    )
    chunks = text_splitter.split_documents(documents)
    print(f"[+] Document split into {len(chunks)} chunks.")

    # Embed chunks and index into Chroma vector store
    embeddings =
OpenAIEmbeddings(model="text-embedding-3-small")
    vector_store = Chroma.from_documents(
        documents=chunks,
        embedding=embeddings,
        collection_name="rag_demo"
    )
    print("[+] Vector store indexed successfully.")

    return vector_store

# 4. Query Execution Loop
def run_interactive_rag_loop(vector_store):
    # Initialize LLM and Prompt Template
    llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.0)

    prompt_template = ChatPromptTemplate.from_messages([
        ("system", "You are a helpful assistant. Answer the
user's question using ONLY the provided context below. If the
answer is not contained within the context, respond with 'I
cannot answer this based on the provided
document.'\n\nContext:\n{context}"),
        ("human", "{question}")
    ])

    print("\n" + "=" * 80)

```

```

    print("RAG System Ready! Ask your questions (type 'exit' or
'quit' to stop).")
    print("=" * 80)

    question_count = 0

    while True:
        try:
            user_question = input(f"\nQuestion #{question_count
+ 1}: ").strip()

            if not user_question:
                continue

            if user_question.lower() in ["exit", "quit"]:
                print("Exiting RAG pipeline session.")
                break

            # Step A: Retrieve Top-K Context Chunks
            retriever =
vector_store.as_retriever(search_kwargs={"k": 2})
            retrieved_docs = retriever.invoke(user_question)

            # Step B: Display Retrieved Chunks Explicitly
            print("\n" + "-" * 40 + " RETRIEVED CONTEXT CHUNKS
" + "-" * 40)

            for idx, doc in enumerate(retrieved_docs, start=1):
                clean_content = doc.page_content.replace("\n",
" ").strip()

                print(f"[Chunk {idx}]: {clean_content}")
            print("-" * 106)

            # Step C: Combine Context & Generate Response
            context_str = "\n\n".join([doc.page_content for doc
in retrieved_docs])

            formatted_prompt = prompt_template.format_messages(

```

```

        context=context_str,
        question=user_question
    )

    response = llm.invoke(formatted_prompt)

    # Step D: Print Final Output
    print("\n[GROUNDED ANSWER]:")
    print(response.content)

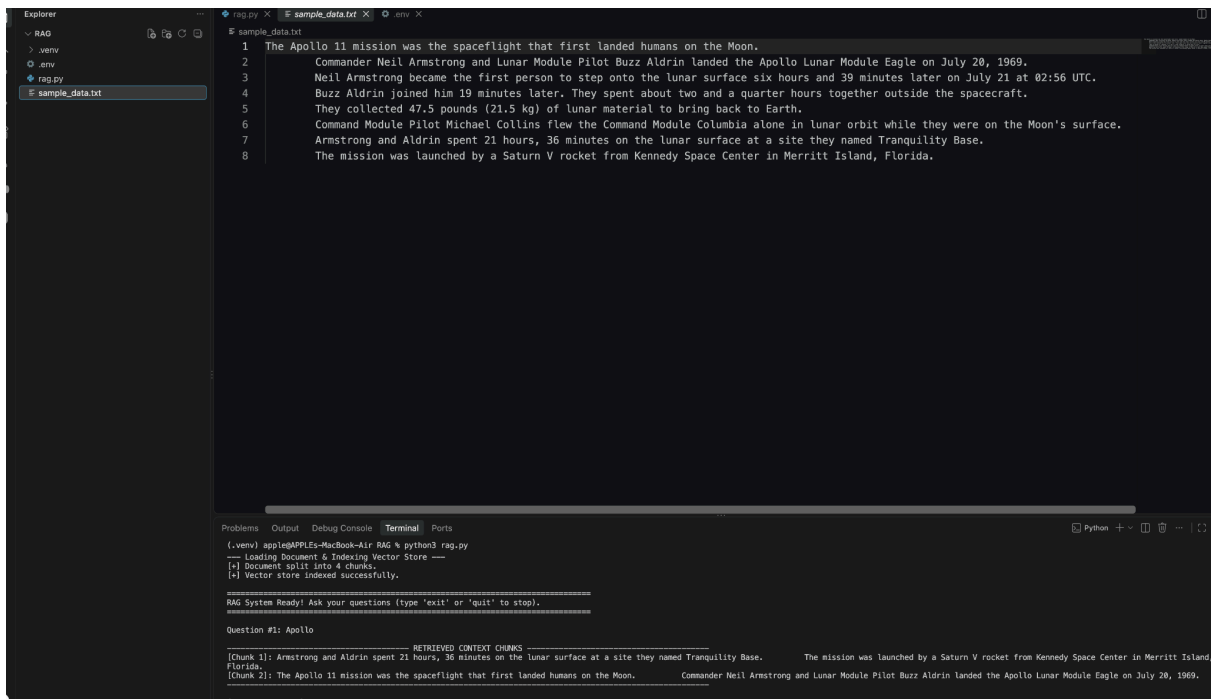
    question_count += 1

except KeyboardInterrupt:
    print("\nSession interrupted. Exiting.")
    break

if __name__ == "__main__":
    create_sample_document()
    vector_db = initialize_rag_pipeline(DOCUMENT_PATH)
    run_interactive_rag_loop(vector_db)

```

Data : sample_data.txt



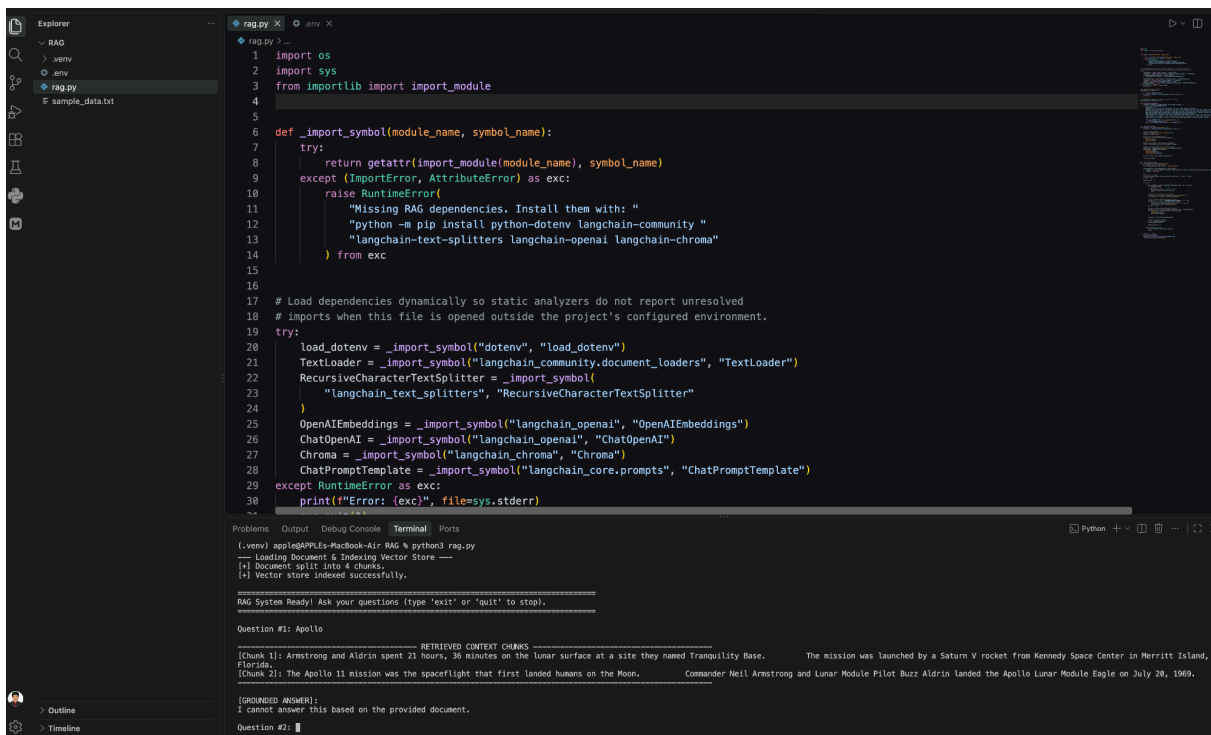
```
1 The Apollo 11 mission was the spaceflight that first landed humans on the Moon.
2 Commander Neil Armstrong and Lunar Module Pilot Buzz Aldrin landed the Apollo Lunar Module Eagle on July 20, 1969.
3 Neil Armstrong became the first person to step onto the lunar surface six hours and 39 minutes later on July 21 at 02:56 UTC.
4 Buzz Aldrin joined him 19 minutes later. They spent about two and a quarter hours together outside the spacecraft.
5 They collected 47.5 pounds (21.5 kg) of lunar material to bring back to Earth.
6 Command Module Pilot Michael Collins flew the Command Module Columbia alone in lunar orbit while they were on the Moon's surface.
7 Armstrong and Aldrin spent 21 hours, 36 minutes on the lunar surface at a site they named Tranquility Base.
8 The mission was launched by a Saturn V rocket from Kennedy Space Center in Merritt Island, Florida.
```

```
(.venv) apple@APPLEs-MacBook-Air: RAG % python3 rag.py
Loading Document & Indexing Vector Store ---
[+] Document split into 4 chunks.
[+] Vector store indexed successfully.

RAG System Ready! Ask your questions (type 'exit' or 'quit' to stop).

Question #1: Apollo

RETRIEVED CONTEXT CHUNKS
[Chunk 1]: Armstrong and Aldrin spent 21 hours, 36 minutes on the lunar surface at a site they named Tranquility Base. The mission was launched by a Saturn V rocket from Kennedy Space Center in Merritt Island, Florida.
[Chunk 2]: The Apollo 11 mission was the spaceflight that first landed humans on the Moon. Commander Neil Armstrong and Lunar Module Pilot Buzz Aldrin landed the Apollo Lunar Module Eagle on July 20, 1969.
```



```
1 import os
2 import sys
3 from importlib import import_module
4
5
6 def _import_symbol(module_name, symbol_name):
7     try:
8         return getattr(import_module(module_name), symbol_name)
9     except (ImportError, AttributeError) as exc:
10         raise RuntimeError(
11             "Missing RAG dependencies. Install them with: "
12             "python -m pip install python-dotenv langchain-community "
13             "langchain-text-splitters langchain-openai langchain-chroma"
14         ) from exc
15
16
17 # Load dependencies dynamically so static analyzers do not report unresolved
18 # imports when this file is opened outside the project's configured environment.
19 try:
20     load_dotenv = _import_symbol("dotenv", "load_dotenv")
21     TextLoader = _import_symbol("langchain_community.document_loaders", "TextLoader")
22     RecursiveCharacterTextSplitter = _import_symbol(
23         "langchain_text_splitters", "RecursiveCharacterTextSplitter"
24     )
25     OpenAIEmbeddings = _import_symbol("langchain_openai", "OpenAIEmbeddings")
26     ChatOpenAI = _import_symbol("langchain_openai", "ChatOpenAI")
27     Chroma = _import_symbol("langchain_chroma", "Chroma")
28     ChatPromptTemplate = _import_symbol("langchain_core.prompts", "ChatPromptTemplate")
29 except RuntimeError as exc:
30     print(f"Error: {exc}", file=sys.stderr)
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

```
(.venv) apple@APPLEs-MacBook-Air: RAG % python3 rag.py
Loading Document & Indexing Vector Store ---
[+] Document split into 4 chunks.
[+] Vector store indexed successfully.

RAG System Ready! Ask your questions (type 'exit' or 'quit' to stop).

Question #1: Apollo

RETRIEVED CONTEXT CHUNKS
[Chunk 1]: Armstrong and Aldrin spent 21 hours, 36 minutes on the lunar surface at a site they named Tranquility Base. The mission was launched by a Saturn V rocket from Kennedy Space Center in Merritt Island, Florida.
[Chunk 2]: The Apollo 11 mission was the spaceflight that first landed humans on the Moon. Commander Neil Armstrong and Lunar Module Pilot Buzz Aldrin landed the Apollo Lunar Module Eagle on July 20, 1969.

[GROUNDED ANSWER]:
I cannot answer this based on the provided document.

Question #2:
```